

```

{
    package main

    var groupStack Stack[Component] = newStack[Component]()
    var documentBuilder IDocumentBuilder = newDocumentBuilder()
    var argumentStack Stack[string] = newStack[string]()
}

/**
 * This section deals with the overall formatting of the document.
 * This includes enforcing an expected format is used, and handling of every
component of the document.
 * Additionally, it is necessary to return the overall Document struct created
to represent the parsed document.
 */
// Matches against overall document structure
Input <- &{
    // re-initialise data
    groupStack = newStack[Component]()
    documentBuilder = newDocumentBuilder()
    argumentStack = newStack[string]()
    return true, nil
} text prerequisite:PrerequisiteContent text docclass:DocumentClass text
preamble:Preamble text content:DocumentContent text EOF {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    // handling prerequisite
    if prerequisite != nil {
        for _, component := range prerequisite.([]Component) {
            errFn := documentBuilder.addPrerequisiteLine(component)
            if errFn != nil {
                return nil, errFn(*startCoord, *endCoord)
            }
        }
    }

    // handling document class
    if docclass != nil {
        // add to the document
        errFn := documentBuilder.addDocumentClass(*docclass.(*Line))
        if errFn != nil {
            return nil, errFn(*startCoord, *endCoord)
        }
    }

    // handling preamble
    if preamble != nil {
        for _, line := range preamble.([]Line) {
            var component Component = &line
            errFn := documentBuilder.addPreambleLine(component)
            if errFn != nil {
                return nil, errFn(*startCoord, *endCoord)
            }
        }
    }

    // handling document content
    if content != nil {
        for i, component := range content.([]Component) {
            var errFn CreateError

```

```

        if i + 1 < len(content.([]Component)) {
            errFn = documentBuilder.addPreambleLine(component)
        } else {
            errFn = documentBuilder.addDocumentContent(component)
        }
        if errFn != nil {
            return nil, errFn(*startCoord, *endCoord)
        }
    }
}

// additional checks over the document occur during creation
document, errFn := documentBuilder.buildDocument()
if errFn != nil {
    return nil, errFn(*startCoord, *endCoord)
}
return document, nil
}

// Handles commands that precede the document class
// These should be rarely used in most LaTeX documents
PrerequisiteContent <- lines:MultiLaTeXLines {
    return lines, nil
}

// Handles the document class i.e. overall categorisation of the LaTeX document
DocumentClass <- command_start name:`documentclass` args:MultiArgument {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    // parse data
    arguments := args.([]Argument)
    line := newLine(string(name.([]byte)), arguments, *startCoord, *endCoord)
    return line, nil
} / text {
    return nil, nil
}

// Handles metadata (i.e. the head) about the LaTeX document
Preamble <- lines:MultiSimpleLaTeXLines {
    return lines, nil
}

// Handles the overall content (i.e. the body) of the LaTeX document
DocumentContent <- lines:MultiLaTeXLines {
    return lines, nil
}

// Ensures the document does not match any further
// i.e. it ensures the End-of-File has been reached
EOF <- !.

/**
 * This section deals with LaTeX lines that require grouped contexts.
 * For example, all document content should be grouped under the 'document'.
 */
MultiLaTeXLines <- lines:LaTeXLine* {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
}

```

```

var result []Component
stack := newStack[Component]()
for _, line := range lines.([]any) {
    if line == nil {
        continue
    }

    // Handling ending groups
    coordinate, ok := line.(Coordinate)
    if ok {
        group, err := stack.Pop()
        if err != nil {
            return nil, err(*startCoord, *endCoord)
        }

        // updating group's end
        group.SetEndCoordinate(coordinate)
        continue
    }

    // adding component to pre-existing groups if necessary
    component, ok := line.(Component)
    if !ok || component == nil {
        return nil,
SERVER_RESPONSIBLE_NON_COMPONENT_DATA_STRUCTURE(*startCoord, *endCoord)
    }
    if !stack.IsEmpty() {
        peek, err := stack.Peek()
        if err != nil {
            return nil, err(*startCoord, *endCoord)
        }
        peer, ok := peek.(*Group)
        if !ok {
            return nil,
SERVER_RESPONSIBLE_STACK_CONTAINS_NON_GROUP_ELEMENT(*startCoord, *endCoord)
        }
        peer.AddComponent(component)
        component.SetGroup(peer)

        // add to the result if this is the topmost layer
    } else {
        result = append(result, component)
    }

    // handling grouped contexts
    _, ok = component.(*Group)
    if ok {
        stack.Push(component)
        continue
    }
}
return result, nil
}

LaTeXLine <- text line:(BeginGroup / EndGroup / SimpleLaTeXLine) text {
    return line, nil
}

BeginGroup <- command_start `begin` args:MultiArgument {
    // retrieving useful data
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
}

```

```

}
arguments := args.([]Argument)

// retrieving name of the group
var name string
found := false
for i, arg := range arguments {
    class, ok := arg.(*ClassArgument)
    if ok {
        name = strings.Trim(class.GetValue().(string), WHITESPACE)
        arguments[i] = nil
        found = true
        break
    }
}
if !found {
    return nil, MISSING_BEGIN_GROUP_NAME(*startCoord, *endCoord)
}

// handling groups ordering
arguments = RemoveNilElements[Argument](arguments, nil)
group := newGroup(name, arguments, *startCoord, *endCoord)
groupStack.Push(group)
return group, nil
}

EndGroup <- command_start `end` args:MultiArgument {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    arguments := args.([]Argument)

    // retrieving name of the group
    var name string
    found := false
    for _, arg := range arguments {
        class, ok := arg.(*ClassArgument)
        if ok {
            name = strings.Trim(class.GetValue().(string), WHITESPACE)
            found = true
            break
        }
    }
    if !found {
        return nil, MISSING_END_GROUP_NAME(*startCoord, *endCoord)
    }

    // checking a begin statement exists
    peek, err := groupStack.Peek()
    if err != nil {
        return nil, GROUP_BEGIN_MISSING(*startCoord, *endCoord)
    }

    // checking the group beginning matches its end
    if peek.GetName() != name {
        return nil, GROUP_NAME_MISMATCH(*startCoord, *endCoord)
    }
    groupStack.Pop()

    // requires a non-nil value to indicate it is not an error
    // we supply endCoord so we can update the group coordinates to account for
    all the inner lines

```

```

    return *endCoord, nil
}

/**
 * This section handles simple LaTeX lines that do not require knowing about the
 * other lines to be processed.
 */
MultiSimpleLaTeXLines <- lines:SimpleWrappedLaTeXLine* {
    return AnyInterfaceToTSlice[Line](lines), nil
}

SimpleWrappedLaTeXLine <- text line:SimpleLaTeXLine text {
    return *line.(*Line), nil
}

// Handles any LaTeX line and provides it a data structure to be parsed further
// disallow 'documentclass', 'begin', and 'end' since they are handled
// separately
SimpleLaTeXLine <- command_start !`documentclass` !`begin` !`end` name:word+
args:MultiArgument {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    arguments := args.([]Argument)
    line := newLine(AnyInterfaceToString(name), arguments, *startCoord,
*endCoord)
    return line, nil
}

/**
 * This section deals with argument handling of individual LaTeX commands.
 * Namely, components within them that are surrounded by curly braces '{}' or
 * square brackets '[]'.
 * This section also includes handling for multiple arguments one after the
 * other.
 */

// Handles zero or more arguments in a row, directly one after the other
MultiArgument <- arguments:Argument* {
    return AnyInterfaceToTSlice[Argument](RemoveNilElements[any](arguments.
([]any), nil)), nil
}

// Handles either possible arguments, ensuring a uniform method of management
Argument <- argument:(OptionArgument / ClassArgument) {
    argumentStack = newStack[string]()
    return argument, nil
}

// For our purposes, a class argument is defined as one surrounded by curly
// braces '{}'
ClassArgument <- '{' _ class:ArgumentContent _ '}' {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    if !argumentStack.IsEmpty() {
        return nil, INNER_ARGUMENT_END_MISSING(*startCoord, *endCoord) // custom
err later
    }
    argument := newClassArgument(class.(string))
    return argument, nil
}

```

```

// Handling if the closing brace '}' is missing
} / '{' _ ArgumentContent {
  startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
  if err != nil {
    return nil, DummyError(err)
  }
  return nil, CLASS_ARGUMENT_END_MISSING(*startCoord, *endCoord)
} / ArgumentContent _ '}' {
  startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
  if err != nil {
    return nil, DummyError(err)
  }
  return nil, CLASS_ARGUMENT_START_MISSING(*startCoord, *endCoord)
}

// For our purposes, an option argument is defined as one surrounded by square
brackets '[]'
OptionArgument <- '[' _ options:ArgumentContent _ ']' {
  argument := newOptionArgument(options.(string))
  return argument, nil

// Handling if the closing bracket ']' is missing
} / '[' _ ArgumentContent {
  startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
  if err != nil {
    return nil, DummyError(err)
  }
  return nil, OPTIONS_ARGUMENT_END_MISSING(*startCoord, *endCoord)
} / ArgumentContent _ ']' {
  startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
  if err != nil {
    return nil, DummyError(err)
  }
  return nil, OPTIONS_ARGUMENT_START_MISSING(*startCoord, *endCoord)
}

/**
 * This section handles inner content of arguments. We do not need to parse the
 * data into their own structures, only match against required formatting.
 */

// Handling of the full inner content of an argument
ArgumentContent <- ArgumentPart* {
  return string(c.text), nil
}

// Handling of argument 'edges' such as {} or [], as well as allowing simple
LaTeX lines and ordinary text to pass through
ArgumentPart <- SimpleLaTeXLine / non_argument_content / ArgumentOpen /
ArgumentClose

// Handles opening argument edges, utilising a stack
ArgumentOpen <- str:argument_opener {
  chr := string(str.([]byte))
  argumentStack.Push(chr)
  return chr, nil
}

ArgumentClose <- &{
  // only match against the ending if there is a starting component
  // otherwise, we assume it is to be matched by the argument itself rather
  than its inner contents
  if argumentStack.IsEmpty() {

```

```

        return false, nil
    }
    return true, nil
} str:argument_closer {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    argMapping := map[string]string{
        "[": "]",
        "{": "}",
    }

    closeChr := string(str.([]byte))
    openChr, errFn := argumentStack.Pop()
    if errFn != nil {
        return nil, errFn(*startCoord, *endCoord)
    }
    expectedCloseChr, ok := argMapping[openChr]
    if !ok {
        return nil, INNER_ARGUMENT_END_MISSING(*startCoord, *endCoord)
    }
    if expectedCloseChr != closeChr {
        return nil, INNER_ARGUMENT_MISMATCH(*startCoord, *endCoord)
    }
    return nil, nil
}

/**
 * This section deals with character classes which are used to indicate a group
of characters, or the lack thereof.
 * It serves a purpose of improving maintainability and readability of the
overall PEG script.
 */
// Handles argument edges
argument_opener <- [{}
argument_closer <- [}\]]
non_argument_content <- [^\{\}\]]

// Handles supported commands, currently only including \
command_start <- '\\\
non_special <- [^\]
text "zero or more non-special characters" <- (command_start (& non_whitespace
non_word) / non_special)*

// Handles word characters
word <- [a-zA-Z0-9_]
non_word <- [^a-zA-Z0-9_]

// whitespace handling
non_whitespace <- [^\n\t\r]
whitespace <- [\n\t\r]
_ "zero or more whitespace characters" <- whitespace*

```